Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №8

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в ширину»

Выполнили:
студенты группы 25ВВВ3
Шибанов Ф.Д.
Флеонов М.В.

Пенза 2026

Цель работы и описание алгоритмов

Цель работы: изучить обход графа в ширину, реализовать его для матрицы и списков смежности, самостоятельно построить очередь на связном списке и сравнить время двух реализаций очереди для графов разных порядков. Выполнены основные и все дополнительные задания.

Представление графа

Вершины обозначаются числами от 1 до n. Матрица смежности содержит нули при отсутствии ребра. Для неориентированного графа она симметрична, диагональ нулевая. Внутри программы используются индексы от 0 до n − 1; при выводе номера увеличиваются на единицу.

Списки смежности построены как односвязные списки: узел хранит номер соседней вершины и ссылку на следующий узел. Соседи просматриваются по возрастанию номеров. Преобразование матрицы в списки выполняется один раз до запуска алгоритмов.

Генерация графа

Матрица генерируется функцией rand. В начале программы выполняется srand((unsigned)time(NULL)), чтобы начальное состояние генератора зависело от текущего времени. Для каждой пары разных вершин вероятность наличия ребра задаётся параметром p в процентах. В неориентированном графе значения записываются симметрично. Пользователю не требуется задавать начальное состояние генератора.

Принцип обхода в ширину

Начальная вершина помечается и помещается в FIFO-очередь. Затем вершина извлекается из начала очереди, а её ещё не посещённые соседи помечаются и добавляются в конец. Пометка выполняется при добавлении, поэтому вершина не попадает в очередь повторно. После завершения компоненты поиск возобновляется из первой непосещённой вершины, если нужен полный обход графа.

Среда выполнения

Программа написана на C++. Для сборки достаточно команды g++ main.cpp -o lab8. Запуск и измерения выполнены в Linux x86_64. Порядок графа ограничен 2000 вершинами для ограничения объёма матрицы.

Задание 1 Генерация и обход графа

Пункты 1 и 2 Матрица и стандартная очередь

Сгенерирована симметричная матрица неориентированного невзвешенного графа. Начальная вершина выбирается параметром --start. В реализации используется требуемая методичкой std::queue<int>: push добавляет вершину, front возвращает первую, pop удаляет её.

Пункт 3 со звёздочкой Списки смежности

Вместо просмотра всей строки матрицы просматриваются узлы списка текущей вершины. Правила пометки и работы очереди сохраняются. Программа выводит исходную матрицу, списки и порядки всех трёх реализаций.

```
$ ./lab8 --n 6 --p 40
ЛР8. Обход в ширину. n=6, p=40%, start=1
Матрица смежности:
       1   2   3   4   5   6
  1    0   1   0   1   0   0
  2    1   0   0   0   1   1
  3    0   0   0   0   1   1
  4    1   0   0   0   0   0
  5    0   1   1   0   0   0
  6    0   1   1   0   0   0
Списки смежности:
1: 2 4 -> NULL
2: 1 5 6 -> NULL
3: 5 6 -> NULL
4: 1 -> NULL
5: 2 3 -> NULL
6: 2 3 -> NULL
BFS матрица / std::queue: 1 2 4 5 6 3
Компонент связности: 1
BFS матрица / своя очередь: 1 2 4 5 6 3
Компонент связности: 1
BFS списки / std::queue: 1 2 4 5 6 3
Компонент связности: 1
```

Рисунок 1 — Матрица, списки и результаты трёх вариантов BFS

При n = 6, p = 40% и старте из вершины 1 получен одинаковый порядок 1 2 4 5 6 3. Число компонент связности равно 1. Все три реализации охватывают все вершины графа.

Задание 2 Собственная очередь

Пункт 1 со звёздочкой Очередь на связном списке

Узел содержит номер вершины и ссылку на следующий узел. Очередь хранит начало head и конец tail. При добавлении новый узел присоединяется к tail; в пустой очереди он становится также head. При извлечении head сдвигается на следующий узел, а удалённый узел освобождается. Если очередь опустела, tail обнуляется.

Для задания создан собственный односвязный список с операциями добавления и удаления узла, необходимыми для FIFO-очереди. Матричный BFS отличается от стандартного варианта только операциями очереди.

Проверка несвязного графа

```
$ ./lab8 --n 4 --p 0 --start 3
ЛР8. Обход в ширину. n=4, p=0%, start=3
Матрица смежности:
      1   2   3   4
  1   0   0   0   0
  2   0   0   0   0
  3   0   0   0   0
  4   0   0   0   0
Списки смежности:
1: -> NULL
2: -> NULL
3: -> NULL
4: -> NULL
BFS матрица / std::queue: 3 1 2 4
Компонент связности: 4
BFS матрица / своя очередь: 3 1 2 4
Компонент связности: 4
BFS списки / std::queue: 3 1 2 4
Компонент связности: 4
```

Рисунок 2 — Обход четырёх изолированных вершин из вершины 3

При p = 0 рёбра отсутствуют. После вершины 3 внешняя процедура выбирает ещё не посещённые вершины 1, 2 и 4. Все реализации выводят 3 1 2 4, число компонент равно 4. Это проверяет внешний цикл полного обхода, предусмотренный методичкой.

Оценка времени работы

Замеры: среднее 100 обходов; p=20%; Время CPU, мс.

Для каждого порядка генерируется один граф, который используется всеми сравниваемыми реализациями. Генерация, построение списков и вывод исключены из измеряемого участка. Создание рабочей очереди и рабочих массивов внутри алгоритма включено. Повторные запуски усредняются; значения зависят от нагрузки компьютера и не являются универсальными константами.

n	**Матрица и std::queue, мс**	**Матрица и своя очередь, мс**	**Списки и std::queue, мс**
100	0.091850	0.092350	0.031050
300	0.913090	0.998330	0.152770
600	2.900950	2.804680	0.506500
1000	7.900910	7.714250	1.638310

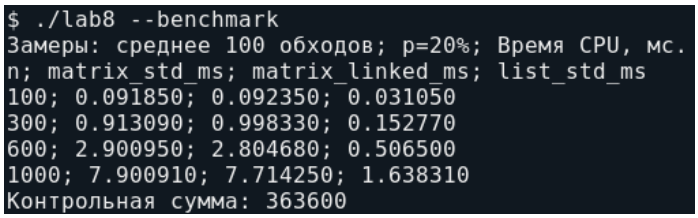


Рисунок 3 — Результаты измерения времени

Для матрицы время BFS составляет $O(n^2)$, для списков — $O(n + m)$, где m — число рёбер. Операции обеих очередей занимают $O(1)$. Собственная очередь создаёт отдельный узел на каждое добавление. Вклад этого выделения памяти может быть мал по сравнению с просмотром матрицы; небольшая разница времени очередей не доказывает устойчивого превосходства одной реализации.

При фиксированном p число рёбер растёт пропорционально n^2, поэтому на таких графах и списочная реализация в пределе имеет квадратичный рост. По эксперименту списки уменьшают стоимость просмотра отсутствующих рёбер.

Базовые проверки и вывод

Базовые проверки

Для графа из четырёх изолированных вершин при старте из вершины 3 получен порядок 3 1 2 4 и число компонент 4. Для графа из одной вершины обход содержит только вершину 1, число компонент равно 1. В полном графе из четырёх вершин при старте из вершины 1 порядок обхода равен 1 2 3 4.

Файлы и запуск

main.cpp содержит алгоритмы и точку входа; graph.h — представление графа, генерацию, ввод, вывод и разбор параметров. В папке также находится готовый исполняемый файл. Для сборки используется одна команда компиляции.

```
g++ main.cpp -o lab8
./lab8 --n 6 --p 40
```

Вывод

Изучен обход графа в ширину. Реализованы все задания, включая представление связными списками и самостоятельную FIFO-очередь. Все способы обходят граф одинаково. Эксперимент показывает влияние представления графа на время и сравнительно небольшой вклад типа очереди при матричном обходе.